October 14, 2026

Agenda

- Sensors: from the analogue world to the digital world
- Analogue-to-digital conversion: sampling and resolution
- Hardware and software interrupts
- The equipment for this session: Arduino Uno and the three sensors of the guide
- Visual programming: a first contact with Node-RED
- Structure of the practical session (2h)

From Theory to Practice

- In Lesson 01 the IoT reference architecture and the M2M paradigm were covered: sensors capture real-world quantities and feed a communication chain up to the *cloud*.
- This lesson closes the first link of that chain: how a microcontroller reads a sensor and how it reacts to events without remaining busy waiting for them.
- These 45 minutes prepare directly the seven exercises of the practical guide for this lesson.

What Is a Sensor (1/2)

- A sensor converts a physical quantity (temperature, humidity, distance, light, ...) into an electrical signal that a digital system can process.
- It may be simple, dedicated to a single quantity, or complex, combining several readings with some pre-processing.
- It is the generic term behind what is informally called “the things” of an IoT system.

What Is a Sensor (2/2)

- **Digital sensors**: they already deliver a discrete reading or use a protocol of their own (for instance the DHT11, which returns values over a one-wire bus).
- **Analogue sensors**: they deliver a continuous voltage, proportional to the quantity measured (for instance the soil moisture sensor, read on pin `A0`).
- The three sensors of the guide cover exactly these two cases, plus one sensor based on echo time (HC-SR04).

Why Convert Analogue into Digital (1/2)

- Microprocessors are digital machines: they only manipulate sequences of bits.
- The physical world is essentially analogue: quantities vary continuously in time.
- An interface circuit between the two worlds is required.

Why Convert Analogue into Digital (2/2)

- The **analogue-to-digital converter** (ADC) transforms the continuous voltage from the sensor into a sequence of numbers that the microcontroller can process.
- The **digital-to-analogue converter** (DAC) performs the reverse path, when it is the digital system that generates a continuous signal.
- In the guide only the ADC is used: it is what lies behind the `analogRead()` function.

Sampling (1/2)

- Sampling consists of measuring the input signal at periodic instants, rather than continuously.
- The set of measured values is only a representation of the original signal, never the exact signal.
- The central question: how often must one measure so as not to lose relevant information?

Sampling (2/2)

- The **Nyquist Theorem** answers: the sampling frequency f_s must be at least twice the fastest frequency component of the signal ($f_s = 2B$).
- In practice a comfortable margin is used, typically 5 to 10 times B.
- For the slow physical quantities of this guide (temperature, humidity, distance), one reading per second is amply sufficient.

Resolution and Quantisation Error

- Besides *when* to measure, it matters *with what precision*: resolution is the number of bits used to represent each sample.
- More bits allow more discrete values and a smaller quantisation error, at the cost of more memory and more conversion time.
- Example: 3 bits distinguish 8 levels; 4 bits distinguish 16, that is, twice the precision.

In Practice: the Arduino Uno ADC

- The Arduino Uno has a 10-bit ADC: each `analogRead()` returns a value between 0 and 1023.
- That value is proportional to the voltage read on the analogue pin, between 0V and the reference voltage (5V by default).
- This is the mechanism used next to read the soil moisture sensor.

Why Interrupts?

- So far, the Arduino `loop()` checks the state of things repeatedly: this is called *polling*.
- Polling works, but it wastes processing cycles and may fail to detect a very short event occurring between two checks.
- An **interrupt** reverses the logic: the processor continues its task and is only diverted when the event actually occurs.

Hardware Interrupts (1/2)

- Certain digital pins of the Arduino Uno (2 and 3) are connected directly to an interrupt controller of the microcontroller.
- An electrical change on that pin, for instance on pressing a button, immediately triggers an **ISR** (*Interrupt Service Routine*), regardless of what the `loop()` was doing.
- It is the most direct mechanism for reacting to fast external events: a button, a limit switch, a pulse from another circuit.

Hardware Interrupts (2/2)

- In code: `attachInterrupt(pin, isrFunction, mode)`, where the mode defines when to trigger (`CHANGE`, `RISING`, `FALLING`).
- In the corresponding exercise, a button connected to pin 2 triggers an ISR that turns the LED on or off, without the `loop()` checking anything at all.
- Note that the `loop()` of the example is practically empty: all the work takes place in the ISR.

Software Interrupts (1/2)

- Not all events come from a pin: sometimes one merely wants something to happen **periodically**, from time to time.
- That is the case of a timer: instead of counting time manually in the `loop()`, an internal peripheral of the microcontroller counts on our behalf and triggers an interrupt when the defined time elapses.
- On the Arduino Uno this is available through the `TimerOne` library.

Software Interrupts (2/2)

- In code: `Timer1.initialize(us)` sets the period (in microseconds) and `Timer1.attachInterrupt(function)` attaches the ISR to that timer.
- In the corresponding exercise, an ISR running every 10 seconds acts on a counter that the `loop()` also increments, an elementary example of two lines of execution sharing the same variable.
- This exercise prepares directly the ground for the periodic sensor readings that follow.

Good Practice in an ISR

- An ISR should be **short and fast**: while it executes, the rest of the program waits.
- Variables shared between the ISR and the `loop()` must be declared `volatile`, so that the compiler does not keep them only in registers and assume outdated values.
- `delay()` and slow operations inside an ISR should be avoided, the most common mistake of those beginning to use interrupts.

The Equipment for This Session (1/2)

- **Arduino Uno**: an open microcontroller platform, with hardware and software licensed under GPL/LGPL, the basis of all the exercises.
- **Onboard LED (pin 13) and push button**: for the two interrupt exercises.
- **Soil moisture sensor (Funduino)**: a resistive analogue sensor, read on `A0`.

The Equipment for This Session (2/2)

- **DHT11**: a digital temperature and humidity sensor, with a one-wire protocol of its own, which is why it requires a dedicated library (`dht.h`).
- **HC-SR04**: an ultrasonic distance sensor; it measures the echo time of a sound pulse to compute the distance to an obstacle.
- The guide works only with this set, which is the one available in the laboratory.

Assembly Precautions

- Circuits must be assembled with the Arduino **disconnected from USB**, or at least with no program running that activates the pins involved.
- The polarity of the sensors must be confirmed before powering: VCC and GND swapped may damage the component.
- No prior experience of electronics is required: each exercise in the guide states exactly which wires to connect to which pins.

Visual Programming: No-Code and Low-Code

- Not all the logic of an IoT system needs to be written line by line: **no-code** and **low-code** tools allow behaviours to be assembled by connecting visual blocks.
- They do not replace programming, since the Arduino still requires its program, but they speed up the prototyping of what comes after the sensor: panels, rules and integrations.
- It is an approach particularly common in IoT, where many different pieces (sensors, protocols, databases, notifications) must be connected quickly.

Node-RED

- **Node-RED** is an open-source visual programming tool based on *flows*: nodes are dragged and connected in a browser, without writing JavaScript.
- Each node represents a small function, such as reading a serial port, communicating over MQTT or drawing a chart, and the wires between them represent the path of the data.
- In the laboratory session Node-RED is the main tool: with the `StandardFirmata` firmware loaded on the board, the Arduino pins are read and written directly from the nodes, without writing code.
- Only the sensors with timed protocols (DHT11 and HC-SR04) require a *sketch* of their own, supplied ready to load.
- Node-RED is revisited in Lesson 12, then receiving data over MQTT instead of a serial port.

Structure of the Practical Session (2h)

- Seven exercises, all involving physical assembly: LED, button, soil moisture sensor, DHT11, HC-SR04 and a final panel integrating the several readings.
- The first three are done entirely in Node-RED, without writing code; in the remaining ones, the necessary *sketches* are supplied.
- Exercise 6 returns to the microcontroller level, in order to compare the event-driven model of the visual flows with hardware and software interrupts.

Lesson Summary

- Sensors bridge the analogue world and the digital system; the Arduino Uno ADC samples that quantity in 10 bits, values between 0 and 1023.
- Interrupts, whether by hardware or by software, allow events to be handled without wasting cycles on polling, provided the ISRs are short and the shared variables are declared `volatile`.
- Low-code tools such as Node-RED allow such data to be acquired and displayed without writing code, provided the sensor protocol allows it, a topic taken up again with MQTT in Lesson 12.

Thank you.

Questions and comments